DETAILED PROPOSITION

Level Intelligence OS

A private, agent-native operating layer that captures Level's analog construction, development, hospitality, and capital knowledge, structures messy language, routes agent-assisted decisions, and compounds from Level's own outcomes.

The Pain Before The Platform

The strongest case for Level Intelligence OS starts with the operating friction Level already feels: opportunity signals are scattered, relationship memory is trapped in people and inboxes, RFQs and permits are language-heavy, and executives spend too much attention assembling status instead of making decisions.

Pain Point	What It Looks Like	Why It Matters
Scattered signals	Permits, planning agendas, title changes, broker notes, franchise news, RFQs, and relationship updates arrive in different places.	Promising opportunities can be seen late, duplicated, or missed entirely.
Trapped judgment	Important context lives in individual memory, emails, spreadsheets, and project folders.	Level's advantage is real but hard to search, hand off, measure, or improve.
RFQ and permit friction	Packages, comments, deadlines, and missing requirements are buried in documents and emails.	Teams spend time translating language into tasks and risk can surface late.
Status burden	Leadership often needs to ask what changed, what is stuck, who owns it, and what needs approval.	Executive time gets spent gathering context instead of exercising judgment.

Outcomes, Not Just Code

The deliverable is not a codebase sitting on a shelf. Code is the mechanism; operating outcomes are the product. Success should be measured by visible pipeline, faster triage, clearer approvals, fewer missed follow-ups, and enterprise memory that gets more valuable as Level uses it.

Outcome	What Changes
Pipeline visibility	Active pursuits have owners, stages, scores, next actions, and aging risk in one operating view.
RFQ discipline	Deadlines, missing requirements, scope risk, and go/no-go reasoning are extracted and routed.
Decision velocity	Approvals, exceptions, and weekly executive briefs move leadership from status gathering to judgment.
Institutional memory	Relationships, wins, losses, vendor outcomes, permit delays, and overrides stay inside the enterprise.
Adaptability	Workflows can be added, omitted, or reshaped as Level changes because the operating layer is internal.

Core Thesis	What It Means
Own the operating layer	Level controls the domain model, data, workflows, and roadmap instead of forcing its work into static vendor software.
Start narrow	The first build should focus on opportunity intelligence, RFQ workflow, approval queues, relationship memory, and executive briefs.
Compound over time	Each signal, decision, win, loss, permit delay, vendor outcome, and relationship touch improves the next recommendation.

1. Feasibility Assessment

This is feasible if scoped as an intelligence and operating layer first. It becomes risky only if framed as a full replacement for existing construction, accounting, CRM, or field systems before workflow evidence is gathered.

Question	Assessment
Can this be built?	Yes. The first version can sit above current tools and coordinate signals, RFQs, relationships, approvals, and reporting.
What is the best first wedge?	Opportunity pipeline, relationship memory, RFQ triage, go/no-go memos, approval queue, and weekly executive brief.
What are the main risks?	Data quality, source access, unclear workflow ownership, and adoption. These are solvable through a narrow pilot and human review.

What should not be promised first?	A full Procore, CRM, or accounting replacement. Integrate first; replace selectively only after evidence.

Recommended proof points to add to the client narrative

- A sample weekly executive brief.
- A sample RFQ go/no-go memo.
- A simple ROI calculator using Level's actual volumes.
- A discovery checklist that shows what is needed to launch.
- A risk and mitigation table showing human control over consequential decisions.

2. Operating Architecture

The system should be built as connected layers. Each layer can evolve independently as Level's markets, teams, and workflows change.

Layer	Function	Examples
Sources	Observe external and internal work	Permits, planning, title, RFQs, email, CRM, docs, project systems
Ingestion	Turn raw material into records	Extract entities, dates, owners, deadlines, scopes, risks, confidence
Domain model	Represent Level's business	Accounts, contacts, opportunities, sites, RFQs, bids, permits, approvals
Memory	Preserve structured and unstructured history	Win/loss, relationship notes, vendor outcomes, permit cycles, decision logs
Agents	Prepare and route work	Scout, qualify, summarize, compare, draft, flag, and brief
Workflow and approval	Control actions and risk	Autonomous, suggest, approval required, manual
Interfaces	Make the system usable	Executive home, signal inbox, pipeline, RFQ center, approval queue

3. Opportunity Journey

The client should see a practical operating journey: market movement enters the system, agents prepare the context, humans make controlled decisions, and outcomes improve the next cycle.

Step	What Happens	Value
1. Signal	Permit, planning agenda, title transfer, broker note, brand expansion, or inbound RFQ is detected.	Level sees movement earlier.
2. Enrich	System connects site, owner, market, relationship history, jurisdiction, and similar work.	Context is not trapped in inboxes or memory.
3. Qualify	Opportunity is scored and explained using Level-specific criteria.	Go/no-go decisions become more consistent.
4. Route	The right owner receives the brief and next action.	Work moves without status chasing.
5. Approve	External outreach, bid submissions, vendor awards, budgets, and investor notes require approval.	Agents help without creating risk.
6. Learn	Wins, losses, delays, margin, response time, and overrides are retained.	The system compounds from Level's own history.

4. Agentic Structure and Governance

The system should use specialist agents with bounded tools, permissions, audit logs, and clear escalation rules. The design goal is useful autonomy without uncontrolled business risk.

Agent	Role
Orchestrator	Routes requests, retrieves context, calls specialists, enforces permissions, and logs activity.
Signal Scout	Detects signals, dedupes records, summarizes evidence, and proposes owner and next action.
Qualification	Scores opportunities, explains reasoning, retrieves comparable history, and recommends pursue/watch/pass.
Relationship	Builds account briefs, tracks owners, finds warm intros, flags stale relationships, and drafts outreach.
RFQ/Bid	Summarizes packages, extracts requirements, detects missing documents, and drafts go/no-go memos.
Permit/Entitlement	Tracks statuses, summarizes comments, drafts follow-ups, and flags jurisdictional blockers.
Executive Briefing	Produces weekly briefs, pending decision lists, stuck pursuits, risk summaries, and market movement.

Authority Level	Agent Can Do	Examples
Autonomous	Act and log	Summaries, classification, dedupe, internal briefs, score drafts
Suggest	Propose action	Tasks, stage updates, draft emails, next actions, internal reminders
Approval Required	Prepare but wait for human approval	External outreach, bid submission, vendor award, budget change, investor message
Manual	Assist only	Contracts, financing, legal commitments, final strategic decisions

5. ROI and Value Model

The initial ROI comes from time savings and fewer missed items. The stronger argument is better pursuit, stronger memory, faster decisions, and a proprietary operating dataset.

Value Lever	How Value Is Created	How To Measure
Opportunity capture	Signals become structured opportunities with owner, score, source, and next action.	Signals captured, conversion rate, time to first action, wins by source
Pursuit discipline	Low-fit pursuits are filtered earlier while high-fit work gets faster attention.	Go/no-go cycle time, win rate by score, avoided pursuit hours, margin by source
Admin reduction	Agents summarize, extract, draft, route, and remind across language-heavy workflows.	Hours saved per RFQ, report time saved, overdue task reduction
Risk reduction	Permits, missing docs, stalled decisions, and deadline risk surface sooner.	Permits past expected review, response time, missed deadline rate
Enterprise asset	Relationships, decisions, outcomes, vendors, and market patterns become private memory.	Record completeness, recommendation accuracy, override rate, reuse of history

6. 90-Day Pilot and Term Structure

The pilot should prove usefulness quickly while preserving a path to deeper workflows, integrations, and software ownership.

Period	Focus	Deliverable
Weeks 1-2	Foundation	Data model, imports, roles, active pipeline, account/contact layer, opportunity views
Weeks 3-5	Intelligence	Signal inbox, AI summaries, qualification score, relationship briefs, weekly digest
Weeks 6-8	Workflow	RFQ intake, go/no-go memo, bid deadlines, approval queue, permit tracker foundation
Weeks 9-12	Operating rhythm	Weekly executive brief, stuck-decision reporting, outcome tracking, integration roadmap

Term Area	Recommended Position
Engagement	90-day pilot to design and build Level Intelligence OS v1.
Deliverables	Data model, working interface, agent workflows, pilot report, success metrics, and phase-two roadmap.
Ownership	Level owns its data and generated operating memory. Code/licensing structure to be finalized in agreement.
Commercial options	Fixed-fee pilot, build fee plus monthly support, strategy sprint then implementation, or long-term internal software partnership.

7. Why This Is Convincing

The strongest argument is not that AI can automate tasks. It is that Level can own a living software layer that gets more valuable as the company uses it.

Proof Wedge	Trust Model	Strategic Asset
Pipeline clarity, RFQ triage, relationship briefs, approval queues, and executive reporting.	Agents prepare decisions and explain reasoning. Humans approve money, outreach, vendors, legal, and investors.	The system learns from Level's own opportunities, decisions, outcomes, and relationships.

Recommended next step: run a focused discovery sprint to collect current pipeline, RFQ examples, relationship records, software map, and decision workflow. Use that to finalize the 90-day pilot scope and pricing.